

**ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**

---□&□---



SOICT

BÁO CÁO ĐỒ ÁN

Project 1

**Đề tài: Xây dựng mô hình AI
dự báo có mưa**

GVHD: TS. Lê Tấn Hùng

Sinh viên thực hiện:
Nguyễn Việt Anh 20215307

Hà Nội, tháng 12 năm 2024

1. Mô tả bài toán	3
2. Thu thập dữ liệu	3
3. Tiền xử lý dữ liệu	5
- Các biểu đồ đánh giá tổng quát	7
- Biểu đồ phân tích đặc trưng	7
Kết luận	10
- Tổng Quan Dữ Liệu	10
- Chất Lượng Dữ Liệu	10
4. Xây dựng mô hình và huấn luyện	11
4.1. Đọc dữ liệu và chọn các trường dùng để huấn luyện mô hình	11
4.2 Chia dữ liệu	11
4.3 Lập trình mô hình	12
4.4 Huấn luyện và kiểm tra mô hình	14
5. Đánh giá mô hình	15
5.1. Đánh giá về thuật toán được lựa chọn K-NN	15
a. Ưu điểm của thuật toán K-NN trong dự án	15
b. Nhược điểm của thuật toán K-NN	15
5.2. Đánh giá về sự thay đổi của k và độ chính xác (accuracy)	15
5.3. Các biểu đồ đánh giá mô hình (với k =55)	16
6. Kết luận	20

1. Mô tả bài toán

Dự báo mưa là một phần quan trọng trong lĩnh vực khí tượng, giúp cung cấp thông tin cần thiết cho các hoạt động hàng ngày và phòng chống thiên tai. Đề tài này tập trung vào việc phát triển một mô hình dự báo mưa sử dụng thuật toán K-Nearest Neighbors (KNN), dựa trên các yếu tố khí hậu như nhiệt độ, độ ẩm, tốc độ gió, và mức độ che phủ mây. Mô hình KNN được lựa chọn do tính đơn giản và hiệu quả trong dự báo, giúp xác định khả năng mưa bằng cách tìm kiếm các điều kiện thời tiết tương tự trong quá khứ. Bộ dữ liệu được sử dụng để huấn luyện và kiểm thử mô hình bao gồm thông tin thời tiết lịch sử chi tiết từ nhiều năm, thu thập từ các nguồn dữ liệu trực tuyến đáng tin cậy và cơ sở dữ liệu khí tượng. Kết quả của dự án này không chỉ cung cấp cái nhìn sâu sắc về mối quan hệ giữa các yếu tố thời tiết và lượng mưa, mà còn gợi mở tiềm năng áp dụng các phương pháp dự báo đơn giản nhưng mạnh mẽ trong việc cải thiện dự báo thời tiết và ứng phó với các hiện tượng thời tiết cực đoan trong tương lai.

2. Thu thập dữ liệu

- Xác định mục tiêu, nhu cầu thu thập dữ liệu
- Xác định các nguồn dữ liệu
- Lên kế hoạch thu thập dữ liệu
- Cào dữ liệu
- Lưu trữ dữ liệu
- <https://www.kaggle.com/>

(link lấy data: <https://www.kaggle.com/datasets/khinguynhtin/hanoiweather1990-2024-data-from-visual-crossing>)

Thông tin Dữ liệu:

Bộ dữ liệu có 12,539 dòng và 13 cột, bao gồm các thuộc tính liên quan đến thời tiết của Hà Nội qua nhiều năm.

Các cột chứa thông tin về ngày (**datetime**), nhiệt độ cao nhất (**tempmax**), nhiệt độ thấp nhất (**tempmin**), nhiệt độ trung bình (**temp**), độ ẩm (**humidity**), lượng mưa (**precip**), xác suất mưa (**precipprob**), loại mưa (**precipctype**), tốc độ gió (**windspeed**), hướng gió (**winddir**), độ che phủ mây (**cloudcover**), áp suất khí quyển ở mực nước biển (**sealevelpressure**), và điều kiện thời tiết (**conditions**).

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 12539 entries, 0 to 12538

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	datetime	12539 non-null	object
1	tempmax	12482 non-null	float64
2	tempmin	12482 non-null	float64
3	temp	12482 non-null	float64
4	humidity	12482 non-null	float64
5	precip	11702 non-null	float64
6	precipprob	12482 non-null	float64
7	preciptype	5748 non-null	object
8	windspeed	12539 non-null	float64
9	winddir	12461 non-null	float64
10	cloudcover	12482 non-null	float64
11	sealevelpressure	12440 non-null	float64
12	conditions	12482 non-null	object

dtypes: float64(10), object(3)

memory usage: 1.2+ MB

None

	tempmax	tempmin	temp	humidity	precip \
count	12482.000000	12482.000000	12482.000000	12482.000000	11702.000000
mean	27.742325	21.395153	24.296867	80.854094	4.140026
std	5.825042	4.983392	5.179632	10.377342	14.047949
min	8.000000	3.800000	7.000000	31.100000	0.000000
25%	23.900000	18.000000	20.500000	75.000000	0.000000
50%	29.000000	23.000000	25.500000	82.100000	0.000000
75%	32.200000	25.100000	28.400000	88.300000	1.100000
max	43.100000	34.600000	35.500000	100.000000	331.243000

	precipprob	windspeed	winddir	cloudcover \
count	12482.000000	12539.000000	12461.000000	12482.000000
mean	45.946162	17.292400	128.478734	70.092782
std	49.837389	6.342684	96.404498	25.286634
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	13.000000	60.900000	56.300000
50%	0.000000	16.600000	109.600000	77.300000
75%	100.000000	21.400000	140.000000	90.100000
max	100.000000	79.200000	360.000000	100.000000

	sealevelpressure
count	12440.000000
mean	1010.874076
std	7.020771
min	991.100000
25%	1005.200000
50%	1010.600000
75%	1016.300000
max	1037.200000

3. Tiền xử lý dữ liệu

- Làm đẹp dữ liệu

[64] #Loại bỏ các hàng thiếu dữ liệu & trùng lặp

```
data.dropna(subset=['tempmax', 'tempmin', 'temp', 'humidity'], inplace=True)
data.drop_duplicates(inplace = True)
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 12482 entries, 0 to 12538
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   datetime              12482 non-null object
1   tempmax               12482 non-null float64
2   tempmin              12482 non-null float64
3   temp                 12482 non-null float64
4   humidity              12482 non-null float64
5   precip               11702 non-null float64
6   precipprob           12482 non-null float64
7   preciptype            5748 non-null  object
8   windspeed            12482 non-null float64
9   winddir              12461 non-null float64
10  cloudcover            12482 non-null float64
11  sealevelpressure      12440 non-null float64
12  conditions            12482 non-null object
dtypes: float64(10), object(3)
memory usage: 1.3+ MB
None
```

- Chuẩn hóa dữ liệu

🔴 #Loại bỏ các hàng thiếu dữ liệu & trùng lặp

```
data.dropna(subset=['tempmax', 'tempmin', 'temp', 'humidity', 'winddir', 'cloudcover', 'sealevelpressure', 'preciptype'], inplace=True)
data.drop_duplicates(inplace = True)
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 12419 entries, 0 to 12538
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   datetime              12419 non-null object
1   tempmax               12419 non-null float64
2   tempmin              12419 non-null float64
3   temp                 12419 non-null float64
4   humidity              12419 non-null float64
5   precip               12419 non-null float64
6   precipprob           12419 non-null float64
7   preciptype            12419 non-null int64
8   windspeed            12419 non-null float64
9   winddir              12419 non-null float64
10  cloudcover            12419 non-null float64
11  sealevelpressure      12419 non-null float64
12  conditions            12419 non-null int64
dtypes: float64(10), int64(2), object(1)
memory usage: 1.3+ MB
None
```

```
#Chuyển đổi đối với các biến không có thứ tự

# label encoding cho conditions
data['conditions'], unique_labels = pd.factorize(data['conditions'])
print(data.tail())
```

```

datetime  tempmax  tempmin  temp  humidity  precip  preciprob  \
12534  2024-04-26    36.0    27.0  30.9     80.8     0.0     0.0
12535  2024-04-27    40.0    27.0  32.2     69.9     0.0     0.0
12536  2024-04-28    35.0    27.0  30.4     82.7     0.0     0.0
12537  2024-04-29    34.0    28.0  30.2     85.3     0.0     0.0
12538  2024-04-30    36.0    27.0  31.0     79.9     0.0     0.0

preciprype  windspeed  winddir  cloudcover  sealevelpressure  \
12534      0         18.4    109.0         44.4         1000.9  \
12535      0         16.6     97.3          8.8          999.6
12536      0         22.3     94.0         45.3         1001.9
12537      0         20.5    107.4         72.9         1002.0
12538      0         22.3     82.5         40.2          999.1

conditions
12534      1
12535      2
12536      1
12537      1
12538      1
```

```
import numpy as np
data['preciprype'] = np.where(data['precip'] == 'rain', 1, 0)
print(data.head())
print(data.info())
```

```

datetime  tempmax  tempmin  temp  humidity  precip  preciprob  \
0  1990-01-01    15.9    13.4  14.6     96.3     0.0     0.0
1  1990-01-02    17.4    14.6  16.1     85.9     0.0     0.0
2  1990-01-03    18.2    16.1  16.8     82.5     0.0     0.0
3  1990-01-04    19.6    15.2  16.4     88.2     0.0     0.0
4  1990-01-05    18.1    15.4  16.9     93.8     0.0     0.0

preciprype  windspeed  winddir  cloudcover  sealevelpressure  conditions
0          0         7.2     24.1     100.0         1015.9          0
1          0         7.2     25.0     100.0         1019.0          0
2          0         7.2     25.0     100.0         1019.7          0
3          0         7.2     50.0     100.0         1019.0          0
4          0         3.6     50.0     98.6         1018.5          0
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 12419 entries, 0 to 12538
```

```
Data columns (total 13 columns):
```

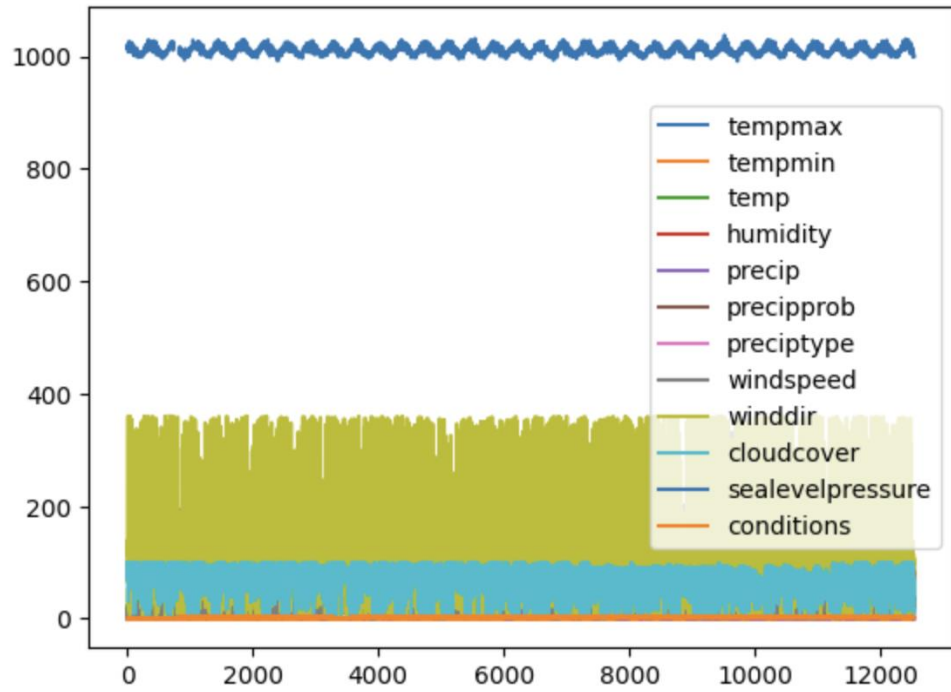
#	Column	Non-Null Count	Dtype
0	datetime	12419 non-null	object
1	tempmax	12419 non-null	float64
2	tempmin	12419 non-null	float64
3	temp	12419 non-null	float64
4	humidity	12419 non-null	float64
5	precip	12419 non-null	float64
6	preciprob	12419 non-null	float64
7	preciprype	12419 non-null	int64
8	windspeed	12419 non-null	float64
9	winddir	12419 non-null	float64
10	cloudcover	12419 non-null	float64
11	sealevelpressure	12419 non-null	float64
12	conditions	12419 non-null	int64

```
dtypes: float64(10), int64(2), object(1)
```

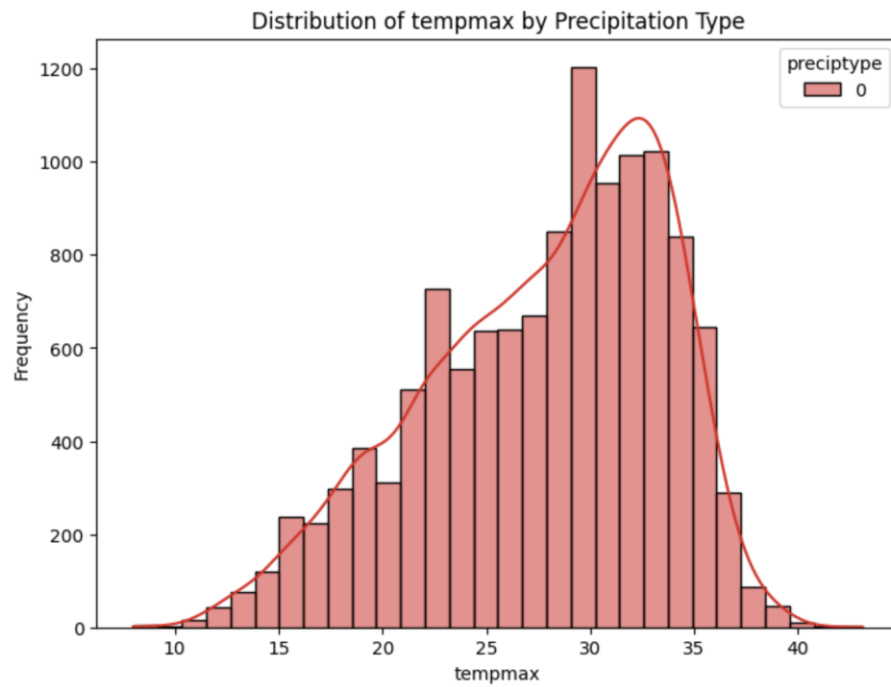
```
memory usage: 1.3+ MB
```

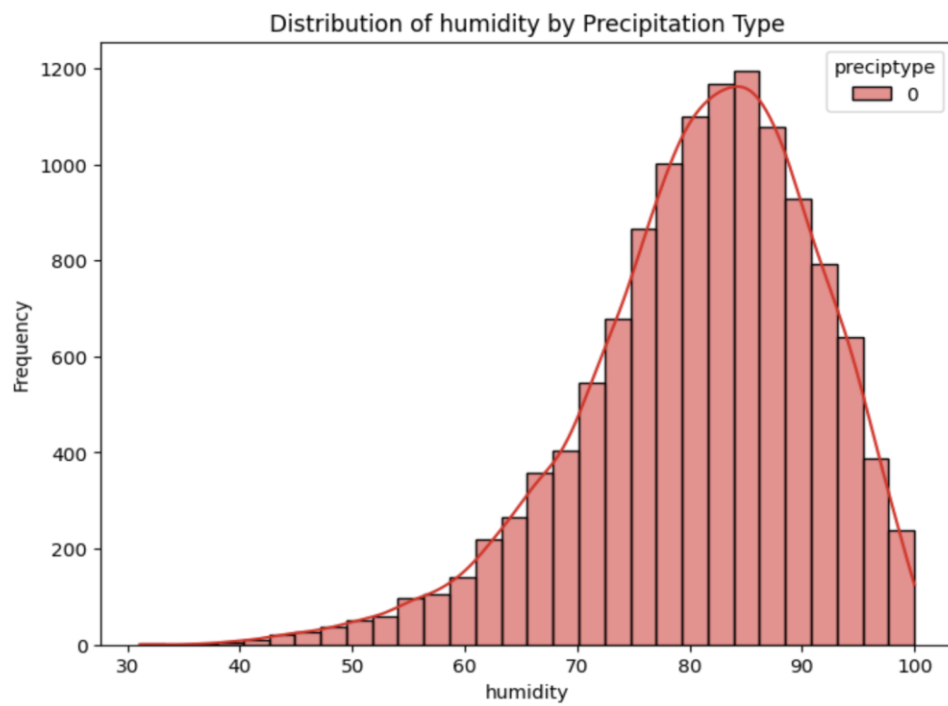
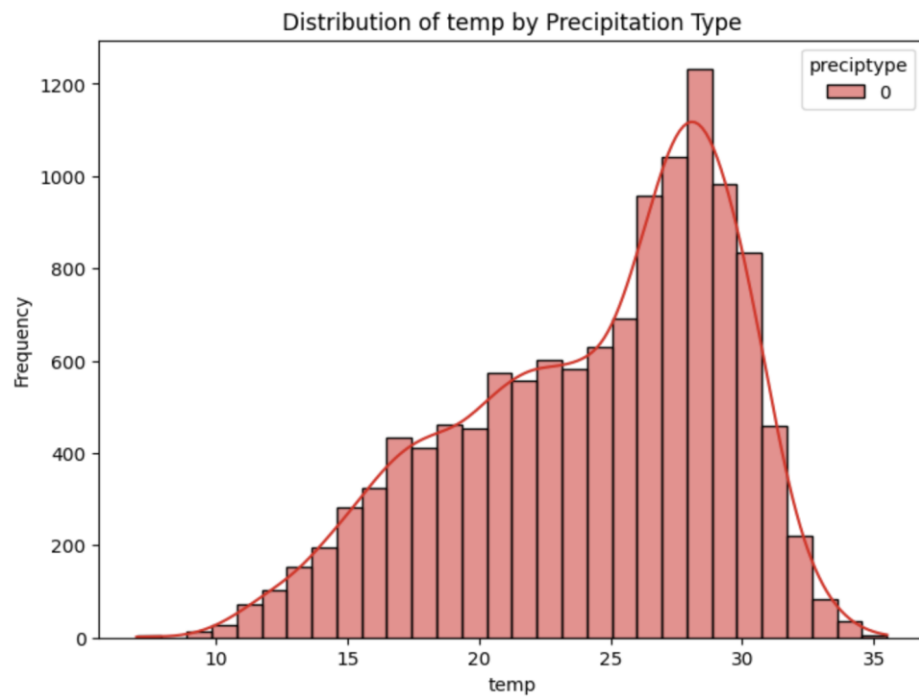
```
None
```

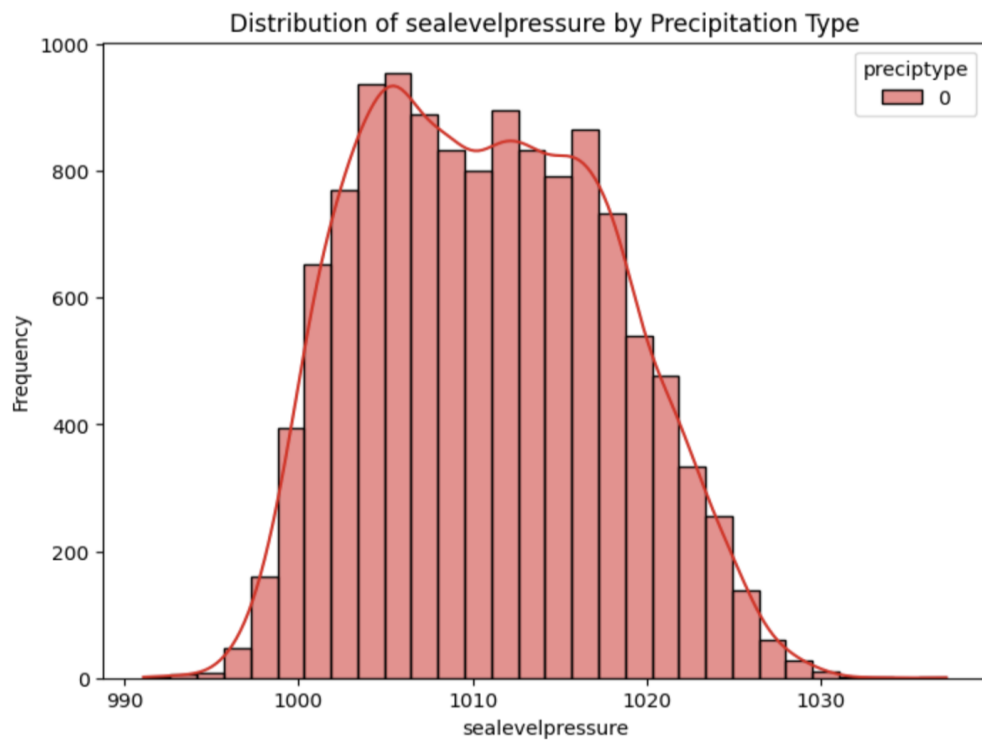
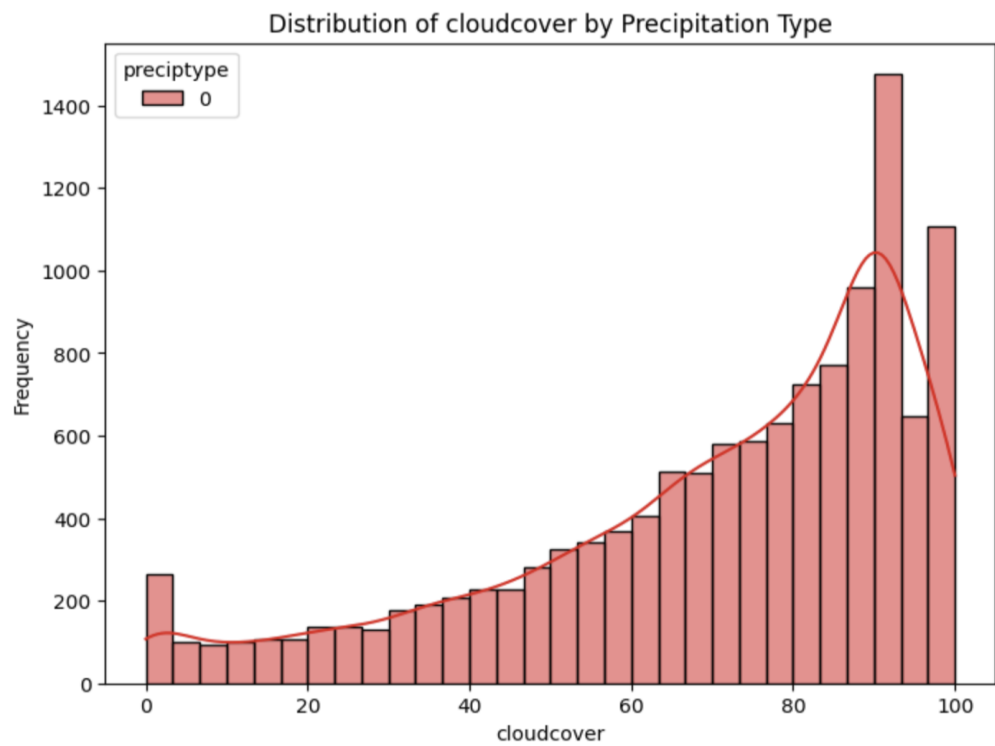
- Các biểu đồ đánh giá tổng quát



- Biểu đồ phân tích đặc trưng







Kết luận

- Tổng Quan Dữ Liệu

Kích Thước Dữ Liệu: Tập dữ liệu hiện tại có 12.419 hàng và 13 cột. Số lượng hàng cho thấy đây là một tập dữ liệu lớn, có thể đủ để thực hiện các phân tích thống kê có ý nghĩa.

Loại Dữ Liệu: Các cột trong tập dữ liệu bao gồm các biến số (như nhiệt độ, độ ẩm) và biến phân loại (như điều kiện thời tiết). Điều này cho phép chúng ta thực hiện các phân tích khác nhau

- Chất Lượng Dữ Liệu

Thiếu Dữ Liệu: Tất cả các ô NaN đã được loại bỏ hoặc thay thế bằng giá trị 0 (đối với cột precip). Điều này đảm bảo rằng không có giá trị thiếu sẽ ảnh hưởng đến kết quả phân tích.

Giá Trị Bất Thường: Đã thực hiện kiểm tra và loại bỏ các giá trị ngoại lệ trong các cột nhiệt độ, giúp dữ liệu trở nên đáng tin cậy hơn cho phân tích.

Biến Đã Được Chuyển Đổi: Cột 'precip_type' đã được dán nhãn với hai giá trị là '0' (không có mưa) và '1' (có mưa). Cột 'conditions' được dán nhãn không thứ tự, thể hiện theo sự đặc trưng của từng giá trị.

Biểu Đồ Phân Phối: Nhìn vào biểu đồ, ta thấy nhiệt độ có xu hướng tập trung vào một khoảng nhất định, cho thấy tính đồng nhất trong dữ liệu.

Khả Năng Phân Tích: Với dữ liệu đã được tiền xử lý, ta có thể tiến hành phân tích thống kê để tìm hiểu mối quan hệ giữa các yếu tố như nhiệt độ và độ ẩm đối với khả năng xuất hiện mưa.

4. Xây dựng mô hình và huấn luyện

4.1. Đọc dữ liệu và chọn các trường dùng để huấn luyện mô hình

- Nhóm lựa chọn các trường đặc trưng dưới đây để huấn luyện mô hình:
 - tempmax: nhiệt độ cao nhất trong ngày
 - tempmin: nhiệt độ thấp nhất trong ngày
 - temp: nhiệt độ trung bình
 - humidity: độ ẩm
 - winddir: hướng gió
 - cloudcover: độ che phủ mây
 - sealevelpressure: áp suất khí quyển ở mực nước biển
 - preciptype: dùng để xác định có mưa hay không (1: mưa, 0: không mưa)

```
# Đọc dữ liệu và chuẩn bị tập dữ liệu
data = pd.read_csv("clean_data.csv")
features = ['tempmax', 'tempmin', 'temp', 'humidity', 'winddir', 'cloudcover', 'sealevelpressure']
X = data[features].dropna().values # Tập dữ liệu đầu vào
y = data['precipitype'].dropna().values # Nhãn đầu ra
```

4.2 Chia dữ liệu

- Nhóm lựa chọn chia dữ liệu thành tập huấn luyện và tập kiểm tra với tỷ lệ 80 - 20.
 - X_train: Là tập dữ liệu đầu vào (features) dùng để huấn luyện mô hình. Mỗi phần tử trong X_train là một vector đặc trưng của một mẫu dữ liệu (ví dụ: nhiệt độ, độ ẩm, áp suất, v.v.).
 - y_train: Là tập nhãn (labels) tương ứng với X_train. Mỗi phần tử trong y_train là một giá trị đầu ra (dự báo mưa hoặc không mưa) liên kết với một mẫu dữ liệu trong X_train.
 - X_test: Là tập dữ liệu đầu vào (features) dùng để kiểm tra mô hình. Đây là các mẫu dữ liệu mà mô hình chưa được "thấy" trước đó trong quá trình huấn luyện.
 - y_test: Là tập nhãn (labels) tương ứng với X_test. y_test chứa các giá trị thực tế của dữ liệu kiểm tra, dùng để so sánh với kết quả dự đoán (y_pred) từ mô hình, nhằm đánh giá độ chính xác (accuracy) của mô hình.

```
# Chia dữ liệu thành tập huấn luyện và kiểm tra
X_train, y_train, X_test, y_test = split_data(X, y)
```

```

# Chia dữ liệu ngẫu nhiên
def split_data(X, y, train_size=0.8):
    # Tạo các chỉ số cho X và y
    data = list(zip(X, y))
    random.shuffle(data) # Xáo trộn dữ liệu
    split_idx = int(len(data) * train_size) # Tính chỉ số chia
    train_data = data[:split_idx]
    test_data = data[split_idx:]

    # Tách X và y từ các phần tử trong train_data và test_data
    X_train, y_train = zip(*train_data)
    X_test, y_test = zip(*test_data)

    return list(X_train), list(y_train), list(X_test), list(y_test)

```

4.3 Lập trình mô hình

- Áp dụng thuật toán KNN cho toàn bộ tập dữ liệu kiểm tra (X_test). Nó sử dụng hàm 'predict' để dự đoán nhãn cho từng điểm dữ liệu thử nghiệm trong tập kiểm tra.

```

# Hàm KNN cho toàn bộ tập dữ liệu
def knn(X_train, y_train, X_test, k):
    y_pred = []
    for test_point in X_test:
        prediction = predict(X_train, y_train, test_point, k)
        y_pred.append(prediction)
    return y_pred

```

- Hàm 'predict' thực hiện dự đoán nhãn (label) cho một điểm dữ liệu thử nghiệm (test_point) dựa trên thuật toán KNN. Nó sử dụng tập dữ liệu huấn luyện để tìm k điểm gần nhất với test_point và đưa ra nhãn dự đoán dựa trên nhãn xuất hiện nhiều nhất trong k hàng xóm gần nhất.

```
# Hàm dự đoán nhãn dựa vào KNN
def predict(X_train, y_train, test_point, k):
    distances = []
    # Tính khoảng cách từ test_point đến tất cả các điểm trong X_train
    for i in range(len(X_train)):
        distance = euclidean_distance(X_train[i], test_point)
        distances.append((distance, y_train[i]))

    # Sắp xếp theo khoảng cách tăng dần và lấy k điểm gần nhất
    distances.sort(key=lambda x: x[0])
    k_nearest_neighbors = [label for _, label in distances[:k]]

    # Đếm số lượng nhãn trong k hàng xóm gần nhất
    most_common = Counter(k_nearest_neighbors).most_common(1)
    return most_common[0][0] # Nhãn xuất hiện nhiều nhất
```

- Tính khoảng cách Euclidean giữa hai điểm trong không gian nhiều chiều. Đây là một phương pháp đo khoảng cách phổ biến trong các thuật toán KNN để xác định độ tương đồng giữa các điểm dữ liệu.

```
# Hàm tính khoảng cách Euclidean giữa 2 điểm
def euclidean_distance(point1, point2):
    distance = 0
    for i in range(len(point1)):
        distance += (point1[i] - point2[i]) ** 2
    return math.sqrt(distance)
```

- Hàm accuracy được sử dụng để đánh giá hiệu suất của mô hình dự đoán. Cụ thể, nó tính toán độ chính xác (accuracy) của mô hình bằng cách đo tỷ lệ dự đoán đúng (giá trị dự đoán khớp với giá trị thực tế) trên tổng số mẫu dữ liệu.

```
# Hàm tính độ chính xác
def accuracy(y_true, y_pred):
    correct = sum([1 for true, pred in zip(y_true, y_pred) if true == pred])
    return correct / len(y_true)
acc = accuracy(y_test, y_pred)
```

4.4 Huấn luyện và kiểm tra mô hình

- Thử với $k = 35$:

```
# Huấn luyện và dự đoán với k=35
k = 35
y_pred = knn(X_train, y_train, X_test, k)
```

- Kết quả:

```
# Huấn luyện và dự đoán với k=35
k = 35
y_pred = knn(X_train, y_train, X_test, k)

# Hàm tính độ chính xác
def accuracy(y_true, y_pred):
    correct = sum([1 for true, pred in zip(y_true, y_pred) if true == pred])
    return correct / len(y_true)
acc = accuracy(y_test, y_pred)

# Thông tin mô hình
print(f"Tổng số mẫu dữ liệu: {len(X_train) + len(X_test)}")
print(f"Số mẫu dữ liệu dùng để huấn luyện: {len(X_train)}")
print(f"Số mẫu dữ liệu dùng để kiểm tra: {len(X_test)}")
print(f"Độ chính xác: {acc:.2%}\n\n")
```

```
Tổng số mẫu dữ liệu: 12419
Số mẫu dữ liệu dùng để huấn luyện: 9935
Số mẫu dữ liệu dùng để kiểm tra: 2484
Độ chính xác: 67.35%
```

5. Đánh giá mô hình

5.1. Đánh giá về thuật toán được lựa chọn K-NN

a. Ưu điểm của thuật toán K-NN trong dự án

- **Dễ triển khai và hiểu rõ:** K-NN là thuật toán đơn giản, không yêu cầu nhiều giả định về dữ liệu.
- **Không cần huấn luyện:** K-NN là thuật toán lazy learning, không cần giai đoạn huấn luyện phức tạp, chỉ cần tính khoảng cách khi dự đoán.
- **Hiệu quả cho dữ liệu nhỏ và vừa:** Với tập dữ liệu vừa phải như dự án này, K-NN hoạt động tốt mà không tốn nhiều tài nguyên tính toán.
- **Linh hoạt với các loại dữ liệu:** K-NN có thể áp dụng cho dữ liệu phân loại (như dự đoán "mưa" hay "không mưa") và dữ liệu hồi quy.

b. Nhược điểm của thuật toán K-NN

- **Nhạy cảm với lựa chọn k:** Giá trị k ảnh hưởng mạnh đến độ chính xác. Chọn k nhỏ sẽ làm tăng độ nhiễu, chọn k lớn có thể làm mất đi các đặc trưng cục bộ.
- **Tính toán chậm với dữ liệu lớn:** Thuật toán phải tính khoảng cách với mọi điểm trong tập huấn luyện, gây tốn thời gian khi dữ liệu lớn.
- **Nhạy cảm với outliers (điểm ngoại lai):** Các điểm bất thường có thể ảnh hưởng lớn đến kết quả dự đoán.
- **Không thể tự động tối ưu hóa:** Không có cơ chế tự học; giá trị k phải được lựa chọn thủ công qua thử nghiệm.

5.2. Đánh giá về sự thay đổi của k và độ chính xác (accuracy)

- Tạo các dữ liệu để đánh giá ngay khi thực hiện việc huấn luyện mô hình:
Sử dụng dữ liệu đã lấy theo tỷ lệ training-testing là 80% - 20% để đánh giá và kiểm thử mô hình.
Dữ liệu huấn luyện / dữ liệu kiểm tra: 9935 / 2484.
- Kết quả chạy thử nghiệm mô hình với các khoảng k:



Quan sát từ biểu đồ:

- Khi giá trị k nhỏ ($k=25$), độ chính xác của mô hình đạt khoảng 0.678.
- Khi tăng dần k lên đến 55, độ chính xác tăng nhẹ và **đạt đỉnh tại $k=55$ (~0.682)**.
- Khi k tiếp tục tăng lên 65, 75, 85...115, độ chính xác giảm dần.

Giải thích lý do:

- K nhỏ (Underfitting - chưa khái quát hóa):

Khi k quá nhỏ, thuật toán dựa vào rất ít điểm lân cận để dự đoán. Điều này làm cho mô hình nhạy cảm với nhiễu (outliers), dẫn đến độ chính xác không ổn định. Mô hình có thể bị overfitting với các mẫu cục bộ.

- K lớn (Overfitting - mất chi tiết cục bộ):

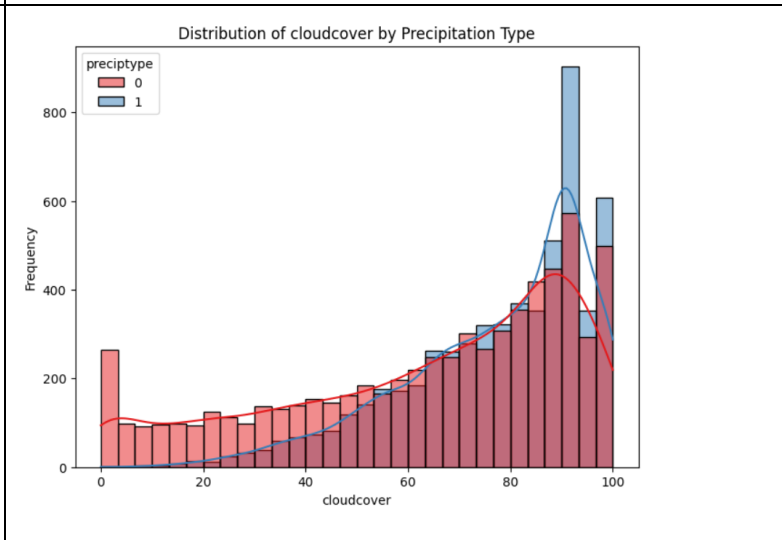
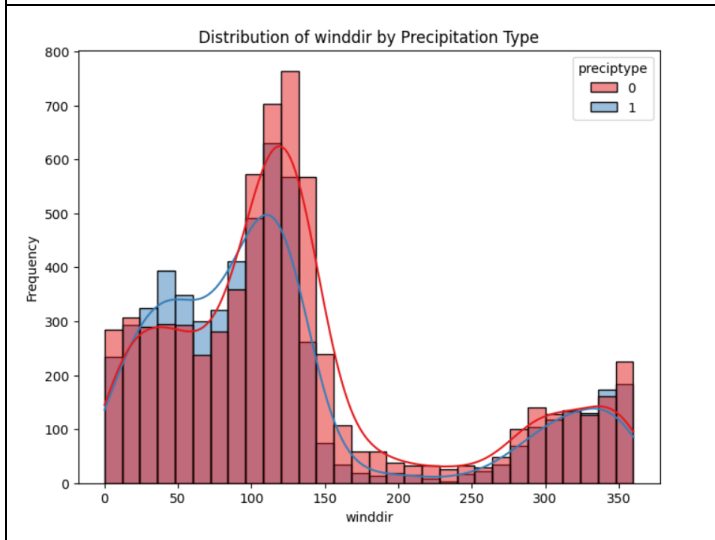
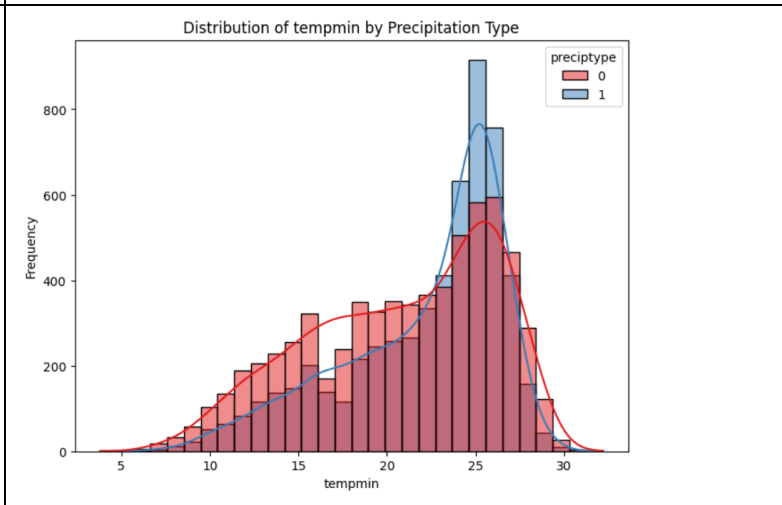
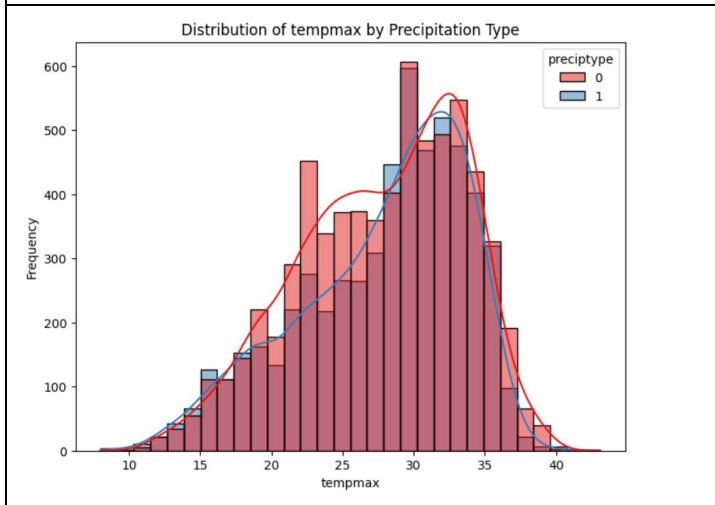
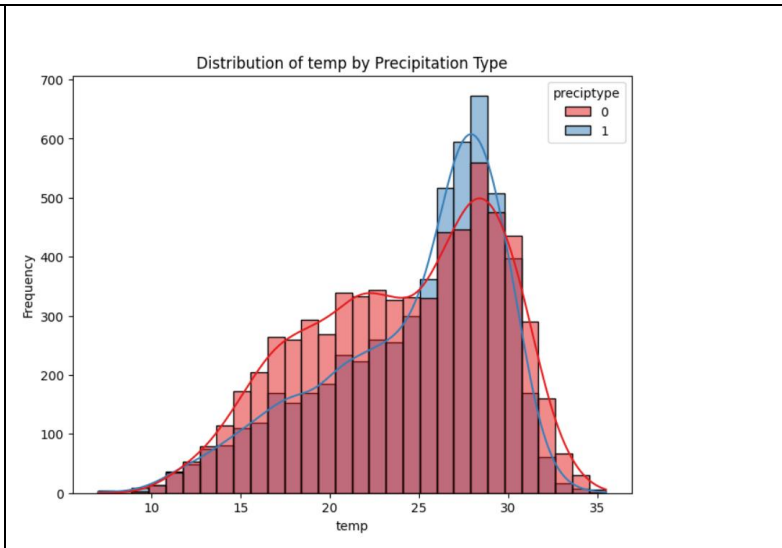
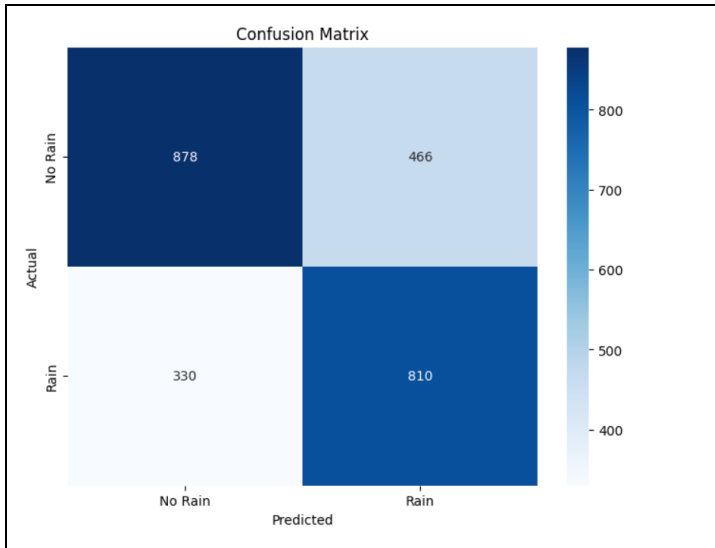
Khi k tăng, K-NN xem xét nhiều hàng xóm hơn. Điều này giúp giảm nhiễu, nhưng có thể làm mất đi các đặc trưng cục bộ.

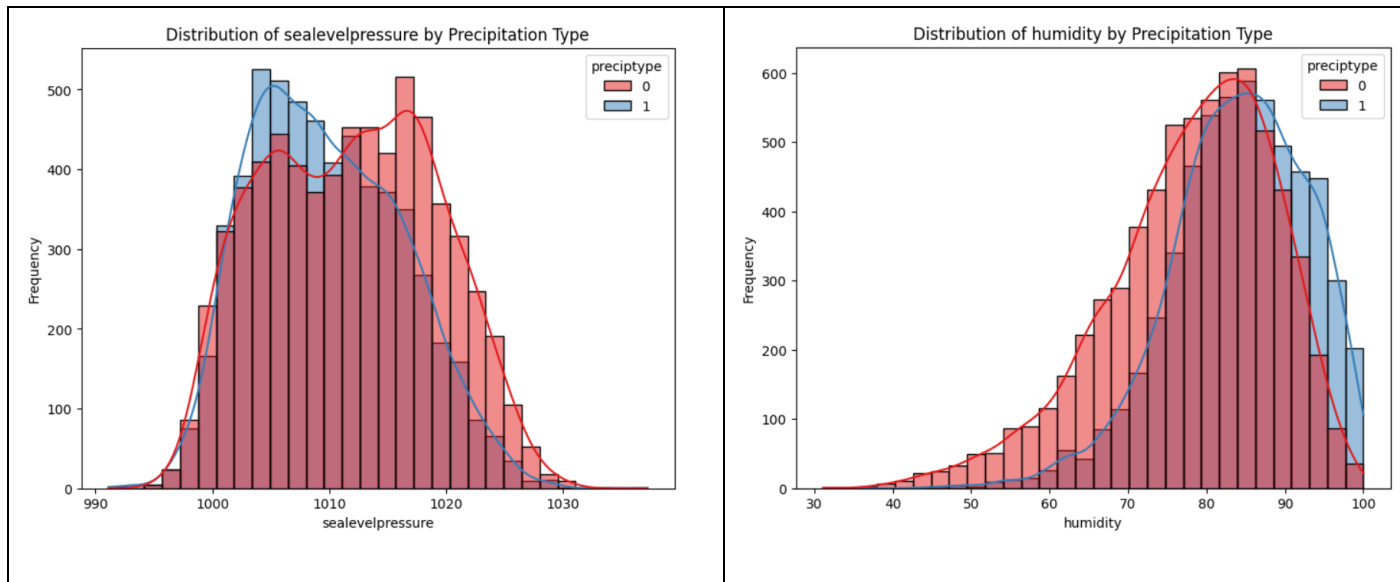
Nếu k quá lớn, mọi điểm sẽ "hòa trộn" và dự đoán bị chi phối bởi nhóm lớn hơn, làm giảm độ chính xác.

- Giá trị k tối ưu:

Độ chính xác đạt cao nhất ở **khoảng $k=55$** .

5.3. Các biểu đồ đánh giá mô hình (với $k=55$)





- Ma trận nhầm lẫn trong hình được tổ chức như sau:

	Predicted: No Rain	Predicted: Rain
Actual: No Rain	878 (TN)	466 (FP)
Actual: Rain	330 (FN)	810 (TP)

TN (True Negative): 878

Số lượng trường hợp thực tế không có mưa và mô hình cũng dự đoán không có mưa.

FP (False Positive): 466

Số lượng trường hợp thực tế không có mưa nhưng mô hình dự đoán có mưa.

FN (False Negative): 330

Số lượng trường hợp thực tế có mưa nhưng mô hình dự đoán không có mưa.

TP (True Positive): 810

Số lượng trường hợp thực tế có mưa và mô hình dự đoán đúng là có mưa.

- Từ ma trận nhầm lẫn, ta có thể tính toán các chỉ số quan trọng sau:

1. **Độ chính xác (Accuracy):**

Đo lường tổng tỷ lệ dự đoán đúng của mô hình.

$$\text{Accuracy} = \frac{TP + TN}{\text{Tổng số mẫu}}$$

- Trong trường hợp này:

$$\text{Accuracy} = \frac{810 + 878}{810 + 878 + 466 + 330} = \frac{1688}{2484} \approx 0.68 \text{ (68\%)}$$

2. **Precision (Độ chính xác cho lớp "Rain"):**

Tỷ lệ dự đoán "Rain" đúng trên tổng số dự đoán "Rain".

$$\text{Precision} = \frac{TP}{TP + FP}$$

- $\text{Precision} = \frac{810}{810+466} \approx 0.635 \text{ (63.5\%)}$

3. **Recall (Độ nhạy hay TPR - True Positive Rate):**

Tỷ lệ mô hình tìm đúng "Rain" trên tổng số thực tế "Rain".

$$\text{Recall} = \frac{TP}{TP + FN}$$

- $\text{Recall} = \frac{810}{810+330} \approx 0.71 \text{ (71\%)}$

6. Xây dựng giao diện tích hợp mô hình

6.1. Tích hợp mô hình trong Backend

- Sau khi huấn luyện và đánh giá mô hình trên nền tảng Colab, tải mô hình về máy tính cá nhân:

```
# Lưu model
joblib.dump(model, '/content/knn_model.joblib')
print("Model saved successfully")

# Tải model về máy
files.download('/content/knn_model.joblib')
```

Model saved successfully

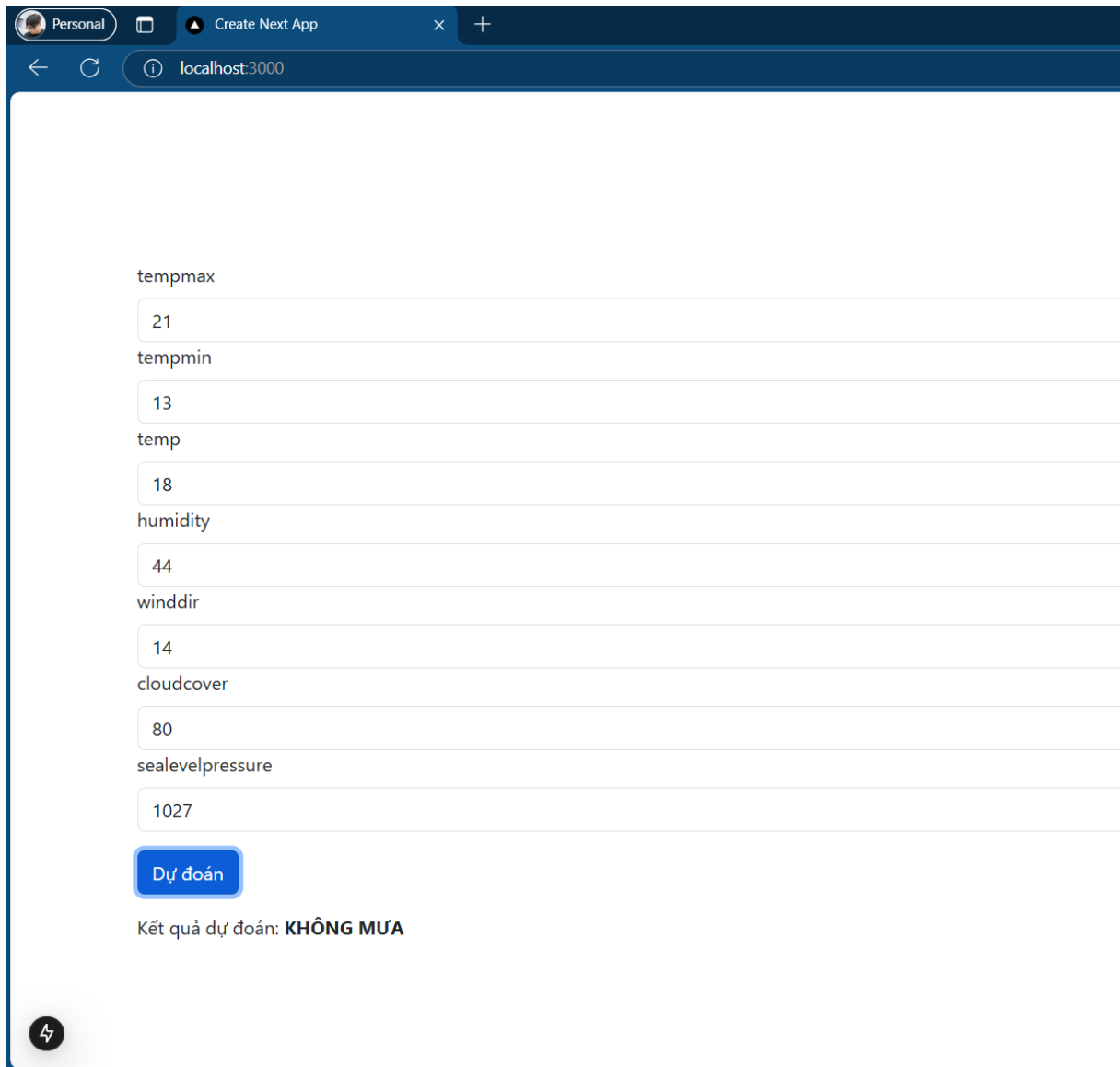
- Sử dụng thư viện FastAPI trong backend để load và sử dụng mô hình:

```
app = FastAPI()

# Cấu hình CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

6.2. Xây dựng Frontend

Xây dựng giao diện cho phép người dùng nhập các chỉ số thời tiết và đưa ra dự báo mưa:



tempmax

21

tempmin

13

temp

18

humidity

44

winddir

14

cloudcover

80

sealevelpressure

1027

Dự đoán

Kết quả dự đoán: **KHÔNG MƯA**

7. Kết luận

Mô hình dự báo mưa sử dụng thuật toán K-Nearest Neighbors (KNN) đã cho kết quả khả quan với độ chính xác gần 70% khi giá trị k tối ưu là 55. KNN là một thuật toán đơn giản, dễ triển khai và không yêu cầu quá trình huấn luyện phức tạp, rất phù hợp với các tập dữ liệu nhỏ và vừa như trong nghiên cứu này. Tuy nhiên, mô hình cũng có một số hạn chế, đặc biệt là khi đối mặt với các giá trị k không tối ưu, gây ra hiện tượng underfitting hoặc overfitting, làm giảm độ chính xác của dự báo.

Mặc dù KNN có thể không phải là mô hình tối ưu nhất trong mọi tình huống, nhưng với các cải tiến như tối ưu hóa giá trị k , loại bỏ các điểm ngoại lai và kết hợp với các phương pháp tiền xử lý dữ liệu, mô hình này vẫn có thể đạt được hiệu suất tốt hơn trong các bài toán dự báo mưa. Việc đánh giá mô hình liên tục và thử nghiệm trên các bộ dữ liệu khác sẽ giúp cải thiện khả năng tổng quát và độ chính xác của mô hình.